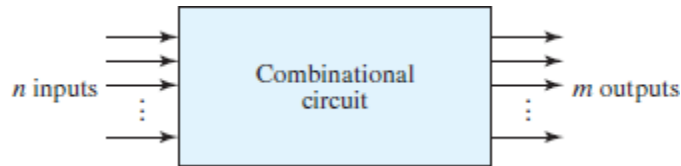


UNIT II

JIT2321 COMBINATIONAL LOGIC

- ❖ *A combinational circuit consists of logic gates whose outputs at any time are determined from only the present combination of inputs.*
- ❖ *A combinational circuit performs an operation that can be specified logically by a set of Boolean functions.*

Sequential circuits:



- ❖ *Sequential circuits employ storage elements in addition to logic gates. Their outputs are a function of the inputs and the state of the storage elements.*
- ❖ *Because the state of the storage elements is a function of previous inputs, the outputs of a sequential circuit depend not only on present values of inputs, but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states.*

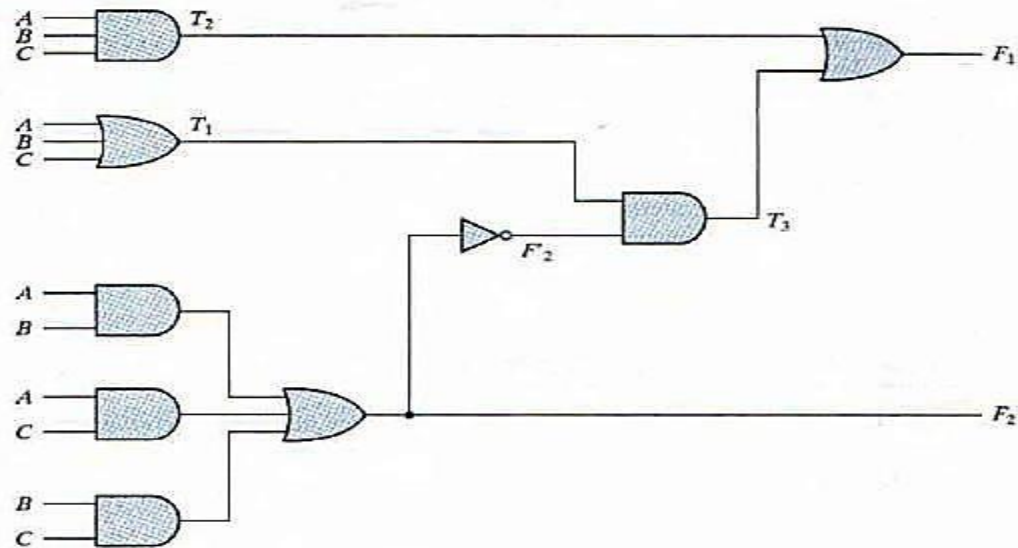
ANALYSIS PROCEDURE

Explain the analysis procedure. Analyze the combinational circuit the following logic diagram.

(May 2015)

- ❖ *The analysis of a combinational circuit requires that we determine the function that the circuit implements.*
- ❖ *The analysis can be performed manually by finding the Boolean functions or truth table or by using a computer simulation program.*
- ❖ *The first step in the analysis is to make that the given circuit is combinational or sequential.*
- ❖ *Once the logic diagram is verified to be combinational, one can proceed to obtain the output Boolean functions or the truth table.*
- ❖ *To obtain the output Boolean functions from a logic diagram,*
 - ✓ *Label all gate outputs that are a function of input variables with arbitrary symbols or names. Determine the Boolean functions for each gate output.*
 - ✓ *Label the gates that are a function of input variables and previously labeled gates with other arbitrary symbols or names. Find the Boolean functions for these gates.*
 - ✓ *Repeat the process in step 2 until the outputs of the circuit are obtained.*
 - ✓ *By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of input variables.*

Logic diagram for analysis example



The Boolean functions for the above outputs are,

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

Next, we consider outputs of gates that are a function of already defined symbols:

$$T_3 = F_2' T_1$$

$$F_1 = T_3 + T_2$$

To obtain F_1 as a function of A , B , and C , we form a series of substitutions as follows:

$$\begin{aligned} F_1 &= T_3 + T_2 = F_2' T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC \\ &= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC \\ &= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC \\ &= A'BC' + A'B'C + AB'C' + ABC \end{aligned}$$

- ❖ Proceed to obtain the truth table for the outputs of those gates which are a function of previously defined values until the columns for all outputs are determined.

Truth Table for the Logic Diagram

A	B	C	F ₂	F ₂ '	T ₁	T ₂	T ₃	F ₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

DESIGNPROCEDURE

Explain the procedure involved in designing combinational circuits.

- ❖ The design of combinational circuits starts from the specification of the design objective and culminates in a logic circuit diagram or a set of Boolean functions from which the logic diagram can be obtained.
- ❖ The procedure involved involves the following steps,
 - ✓ From the specifications of the circuit, determine the required number of inputs and outputs and assign a symbol to each.
 - ✓ Derive the truth table that defines the required relationship between inputs and outputs.
 - ✓ Obtain the simplified Boolean functions for each output as a function of the input variables.
 - ✓ Draw the logic diagram and verify the correctness of the design.

CIRCUITS FOR ARITHMETIC OPERATIONS

Half adder:

Construct a half adder with necessary diagrams.

(Nov-06, May- 07)

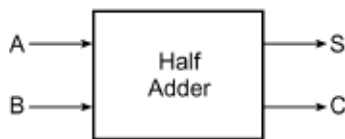
- ❖ A half-adder is an arithmetic circuit block that can be used to add two bits and produce two outputs SUM and CARRY.
- ❖ The Boolean expressions for the SUM and CARRY outputs are given by the equations

$$\text{SUM } S = A.\bar{B} + \bar{A}.B$$

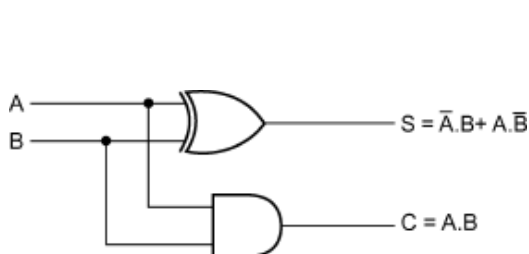
$$\text{CARRY } C = A.B$$

Truth Table:

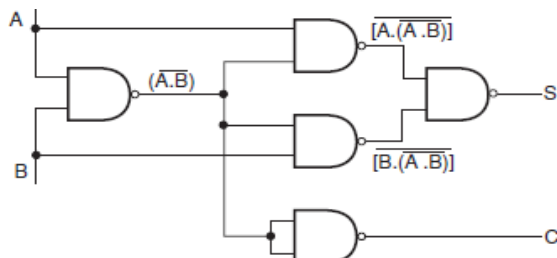
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Logic Diagram:



Half adder using NAND gate:



Full adder:

Design a full adder using NAND and NOR gates respectively.

(Nov -10)

- ❖ A Full-adder is an arithmetic circuit block that can be used to add three bits and produce two outputs SUM and CARRY.
- ❖ The Boolean expressions for the SUM and CARRY outputs are given by the equations

$$S = \overline{A}.\overline{B}.C_{in} + \overline{A}.B.\overline{C}_{in} + A.\overline{B}.\overline{C}_{in} + A.B.C_{in}$$

$$C_{out} = B.C_{in} + A.B + A.C_{in}$$

Truth table:

Input variables			Outputs	
<i>X</i>	<i>A</i>	<i>B</i>	<i>S</i>	<i>C</i>
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Karnaugh map:

	<i>A'B'</i>	<i>A'B</i>	<i>AB</i>	<i>AB'</i>
<i>X'</i>		1		1
<i>X</i>	1		1	

K-Map for Sum

	<i>A'B'</i>	<i>A'B</i>	<i>AB</i>	<i>AB'</i>
<i>X'</i>			1	
<i>X</i>		1	1	1

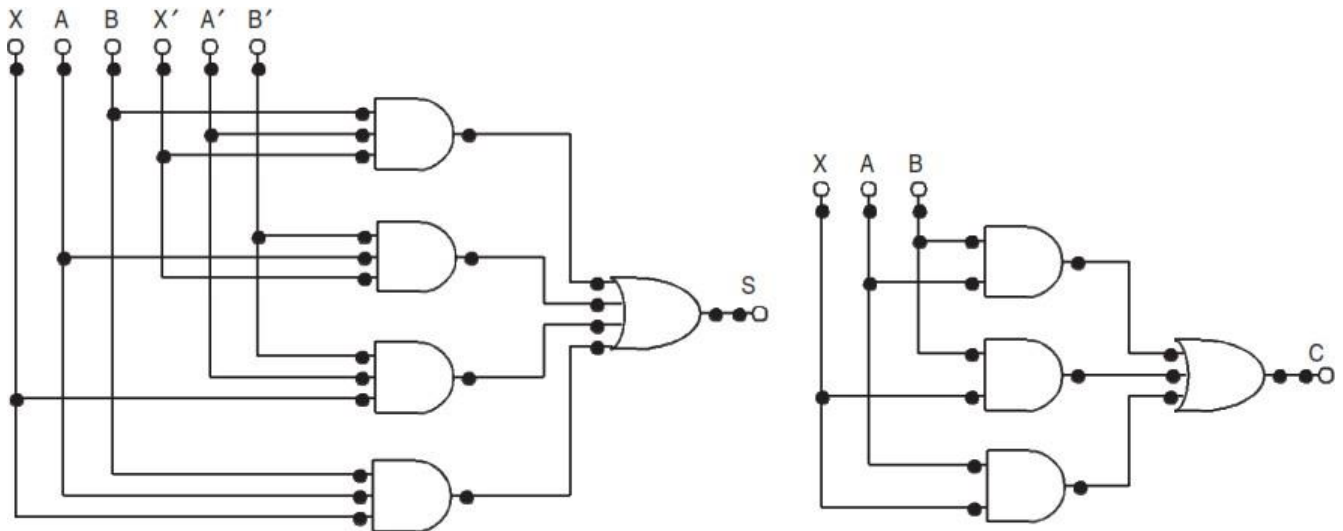
K-Map for Carry

- ❖ The simplified Boolean expressions of the outputs are

$$S = X'A'B + X'AB' + XA'B' + XAB$$

$$C = AB + BX + AX$$

Logic diagram:

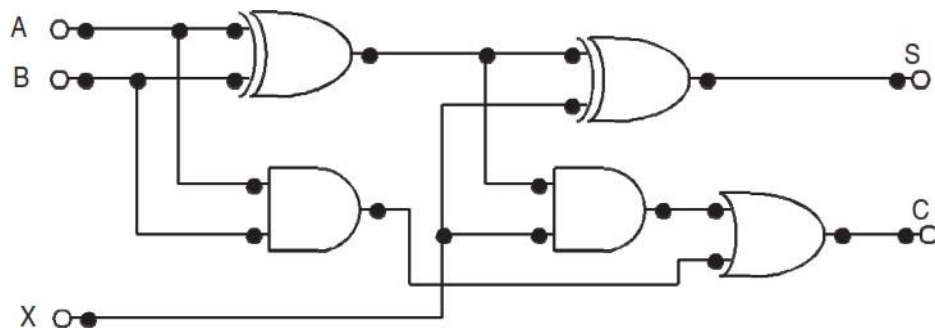


❖ The Boolean expressions of S and C are modified as follows

$$\begin{aligned}
 S &= X'A'B + X'AB' + XA'B' + XAB \\
 &= X' (A'B + AB') + X (A'B' + AB) \\
 &= X' (A \oplus B) + X (A \oplus B)' \\
 &= X \oplus A \oplus B \\
 C &= AB + BX + AX = AB + X (A + B) \\
 &= AB + X (AB + AB' + AB + A'B) \\
 &= AB + X (AB + AB' + A'B) \\
 &= AB + XAB + X (AB' + A'B) \\
 &= AB + X (A \oplus B)
 \end{aligned}$$

Full adder using Two half adder:

❖ Logic diagram according to the modified expression is shown Figure.



Half subtractor:

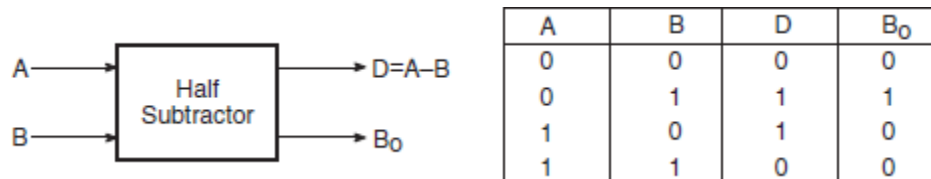
Design a half subtractor circuit.

(Nov-2009)

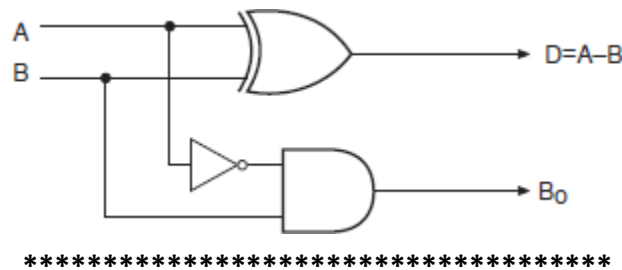
- ❖ A half-subtractor is a combinational circuit that can be used to subtract one binary digit from another to produce a DIFFERENCE output and a BORROW output.
- ❖ The BORROW output here specifies whether a '1' has been borrowed to perform the subtraction. The Boolean expression for difference and borrow is:

$$D = \bar{A}.B + A.\bar{B}$$

$$B_0 = \bar{A}.B$$



Logic diagram:



Full subtractor:

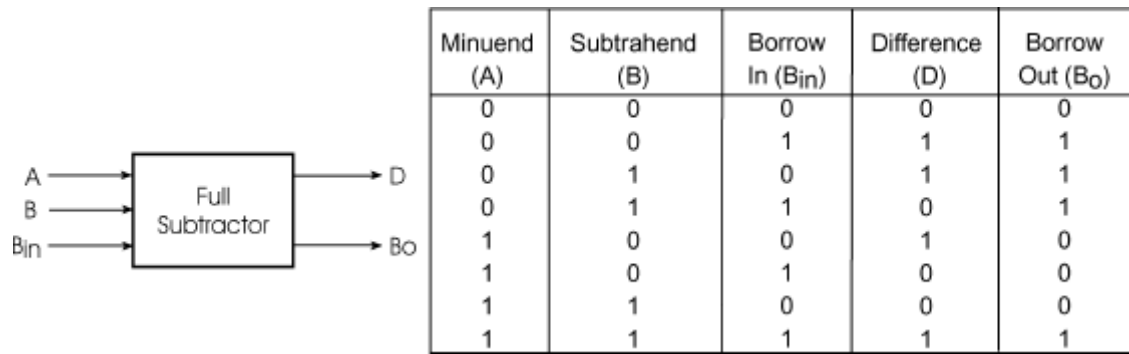
Design a full subtractor.

(Nov-2009,07)

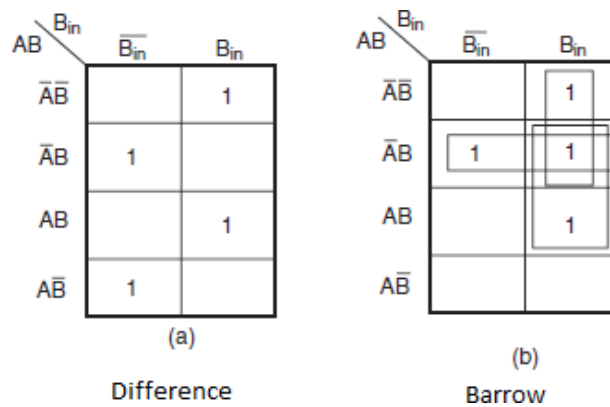
- ❖ A full subtractor performs subtraction operation on two bits, a minuend and a subtrahend, and also takes into consideration whether a '1' has already been borrowed by the previous adjacent lower minuend bit or not.
- ❖ As a result, there are three bits to be handled at the input of a full subtractor, namely the two bits to be subtracted and a borrow bit designated as Bin.
- ❖ There are two outputs, namely the DIFFERENCE output D and the BORROW output Bo. The BORROW output bit tells whether the minuend bit needs to borrow a '1' from the next possible higher minuend bit. The Boolean expression for difference and borrow is:

$$D = \bar{A}.\bar{B}.B_{in} + \bar{A}.B.\bar{B}_{in} + A.\bar{B}.\bar{B}_{in} + A.B.B_{in}$$

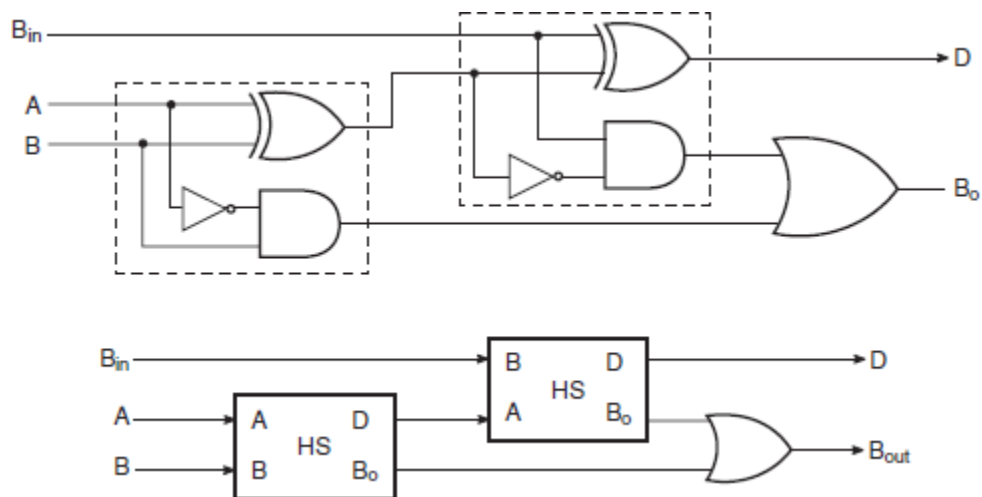
$$B_0 = \bar{A}.B + \bar{A}.B_{in} + B.B_{in}$$



K-Map:



Full subtractor using two half subtractor:



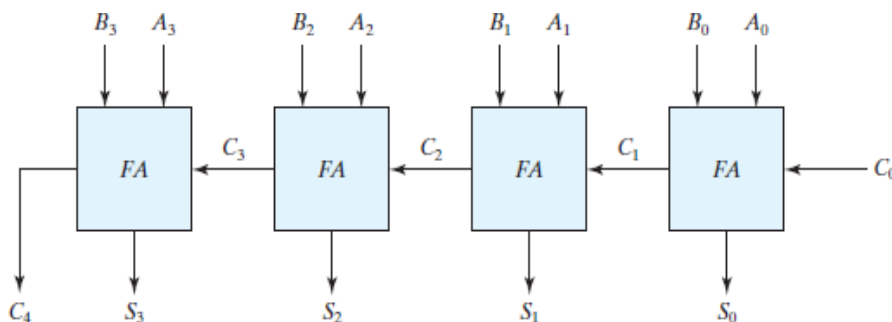
Parallel Binary Adder: (Ripple Carry Adder):

Explain about four bit adder. (or) Design of 4 bit binary adder – subtractor circuit. (Apr – 2019)

- ❖ A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers. It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain.
- ❖ Addition of n-bit numbers requires a chain of n- full adders or a chain of one-half adder and n-1 full adders. In the former case, the input carry to the least significant position is fixed at 0.
- ❖ Figure shows the interconnection of four full-adder (FA) circuits to provide a four-bit binary ripple carry adder.
- ❖ The carries are connected in a chain through the full adders. The input carry to the adder is C_0 , and it ripples through the full adders to the output carry C_4 . The S outputs generate the required sum bits.

Example: Consider the two binary numbers $A = 1011$ and $B = 0011$. Their sum $S = 1110$ is formed with the four-bit adder as follows:

Subscript i:	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}



- ✓ The carry output of lower order stage is connected to the carry input of the next higher order stage. Hence this type of adder is called ripple carry adder.
- ✓ In a 4-bit binary adder, where each full adder has a propagation delay of t_p ns, the output in the fourth stage will be generated only after $4t_p$ ns.
- ✓ The magnitude of such delay is prohibitive for high speed computers.
- ✓ One method of speeding up this process is look-ahead carry addition which eliminates ripple carry delay.

Complement of a number:

1's complement:

The 1's complement of a binary number is formed by changing 1 to 0 and 0 to 1.

Example:

1. The 1's complement of 1011000 is 0100111.
2. The 1's complement of 0101101 is 1010010.

2's complement:

The 2's complement of a binary number is formed by adding 1 with 1's complement of a binary number.

Example:

1. The 2's complement of 1101100 is 0010100
2. The 2's complement of 0110111 is 1001001

Subtraction using 2's complement addition:

- ✓ The subtraction of unsigned binary number can be done by means of complements.
- ✓ Subtraction of $A - B$ can be done by taking 2's complement of B and adding it to A .
- ✓ Check the resulting number. If carry present, the number is positive and remove the carry.
- ✓ If no carry present, the resulting number is negative, take the 2's complement of result and put negative sign.

Example:

Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction

(a) $X - Y$ and (b) $Y - X$ by using 2's complements.

Solution:

(a) $X = 1010100$

2's complement of $Y = + 0111101$

Sum = 10010001

Discard end carry. Answer: $X - Y = 0010001$

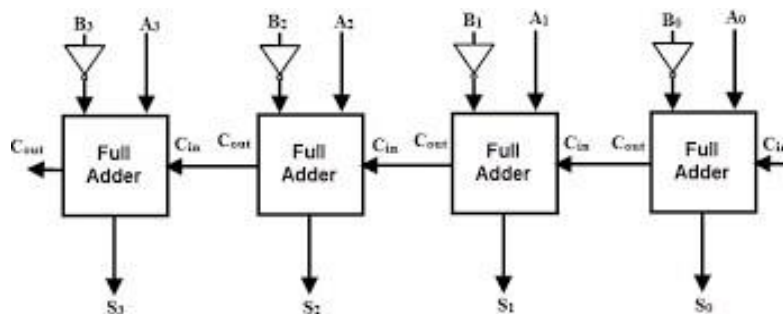
(b) $Y = 1000011$

2's complement of $X = + 0101100$

Sum = 1101111

There is no end carry. Therefore, the answer is $Y - X = -(2's \text{ complement of } 1101111) = -0010001$.

Parallel Binary Subtractor:



- ✓ The subtraction of unsigned binary numbers can be done most conveniently by means of complements. The subtraction $A - B$ can be done by taking the 2's complement of B and adding it to A . The 2's complement

can be obtained by taking the 1's complement and adding 1 to the least significant pair

of bits. The 1's complement can be implemented with inverters, and a 1 can be added to the sum through the input carry.

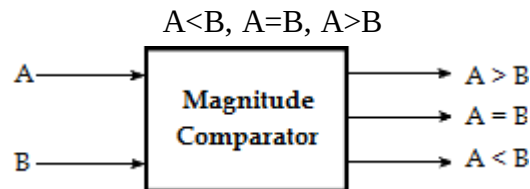
- ✓ The circuit for subtracting $A - B$ consists of an adder with inverters placed between each data input B and the corresponding input of the full adder. The input carry C_{in} must be equal to 1 when subtraction is performed. The operation thus performed becomes A , plus the 1's complement of B , plus 1. This is equal to A plus the 2's complement of B .
 - ✓ For unsigned numbers, that gives $A - B$ if $A \geq B$ or the 2's complement of $B - A$ if $A < B$. For signed numbers, the result is $A - B$, provided that there is no overflow.
-

Comparators

Design a 2 bit magnitude comparator.

(May 2006)

It is a combinational circuit that compares two numbers and determines their relative magnitude. The output of comparator is usually 3 binary variables indicating:



1-bit comparator: Let's begin with 1bit comparator and from the name we can easily make out that this circuit would be used to compare 1bit binary numbers.

A	B	A > B	A = B	A < B
0	0	0	1	0
1	0	1	0	0
0	1	0	0	1
1	1	0	1	0

For a 2-bit comparator we have four inputs A1 A0 and B1 B0 and three output E (is 1 if two numbers are equal) G (is 1 when A > B) and L (is 1 when A < B) If we use truth table and K-map the result is

		A > B	
		0	1
A	B	0	0
	1	1	0

Equation is $A > B = A \cdot \overline{B}$

		A < B	
		0	1
A	B	0	1
	1	0	0

Equation is $A < B = \overline{A} \cdot B$

		A = B	
		0	1
A	B	1	0
	1	0	1

The equation is $f(A=B) = \overline{A} \cdot \overline{B} + A \cdot B$
 $= A \text{ XNOR } B$

Design of 2 – bit Magnitude Comparator.

The truth table of 2-bit comparator is given in table below

Truth table:

Inputs				Outputs		
A ₃	A ₂	A ₁	A ₀	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

K-Map:

For A>B

A ₁ A ₀ \ B ₁ B ₀				
	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	1	0	1
10	1	1	0	0

For A=B

A ₁ A ₀ \ B ₁ B ₀				
	00	01	11	10
00	1	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	1

$$A>B = A_0B_1'B_0' + A_1B_1' + A_1A_0B_0'$$

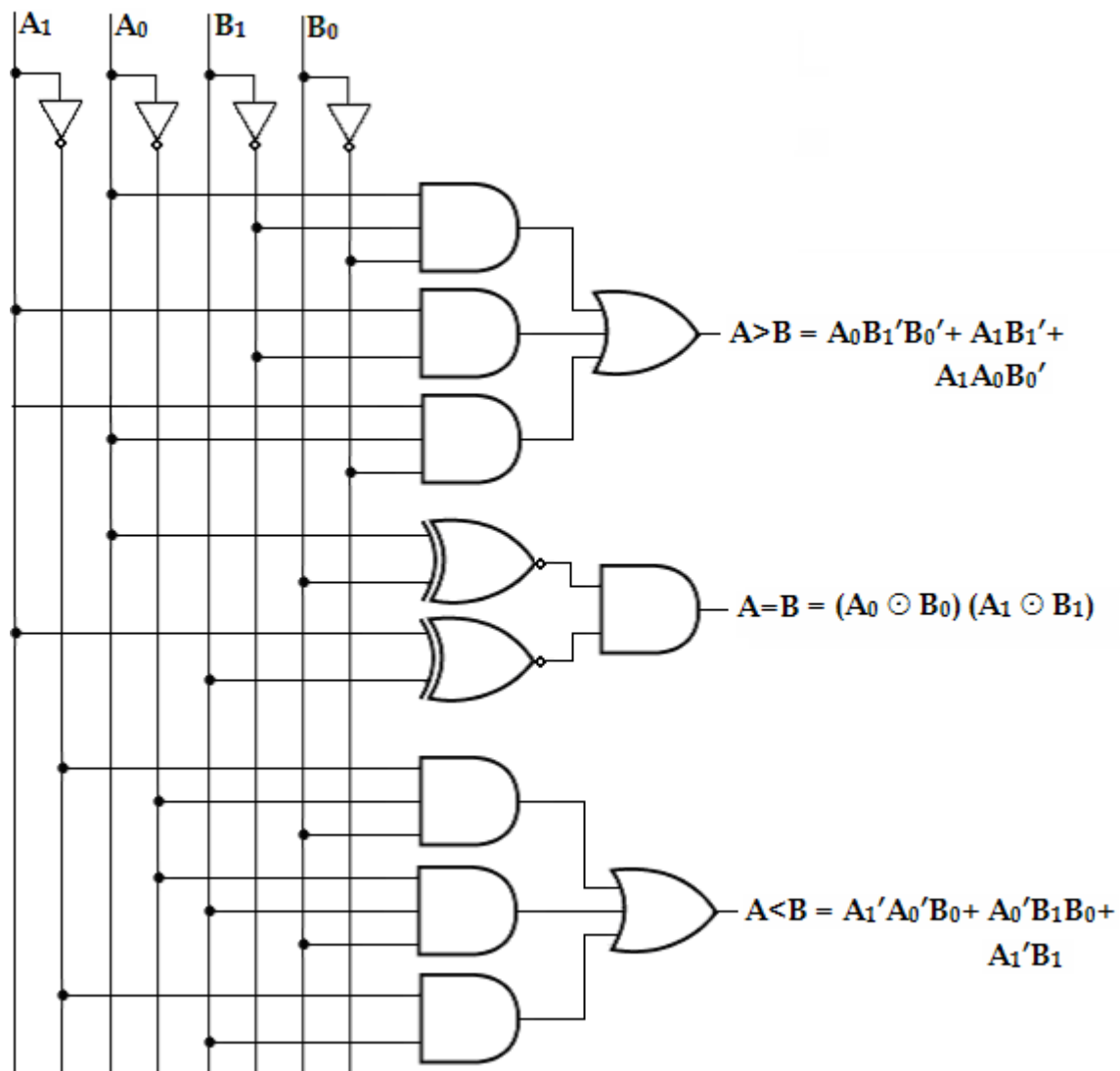
$$\begin{aligned}
 A=B &= A_1'A_0'B_1'B_0' + A_1'A_0B_1'B_0 + \\
 &\quad A_1A_0B_1B_0 + A_1A_0'B_1B_0' \\
 &= A_1'B_1'(A_0'B_0' + A_0B_0) + A_1B_1(A_0B_0 + A_0'B_0') \\
 &= (A_0 \odot B_0)(A_1 \odot B_1)
 \end{aligned}$$

For A<B

A ₁ A ₀ \ B ₁ B ₀				
	00	01	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	1	0

$$A<B = A_1'A_0'B_0 + A_0'B_1B_0 + A_1'B_1$$

Logic Diagram:



4 bit magnitude comparator:

Design a 4 bit magnitude comparators. (Apr – 2019)

Input

$$A = A_3 A_2 A_1 A_0$$

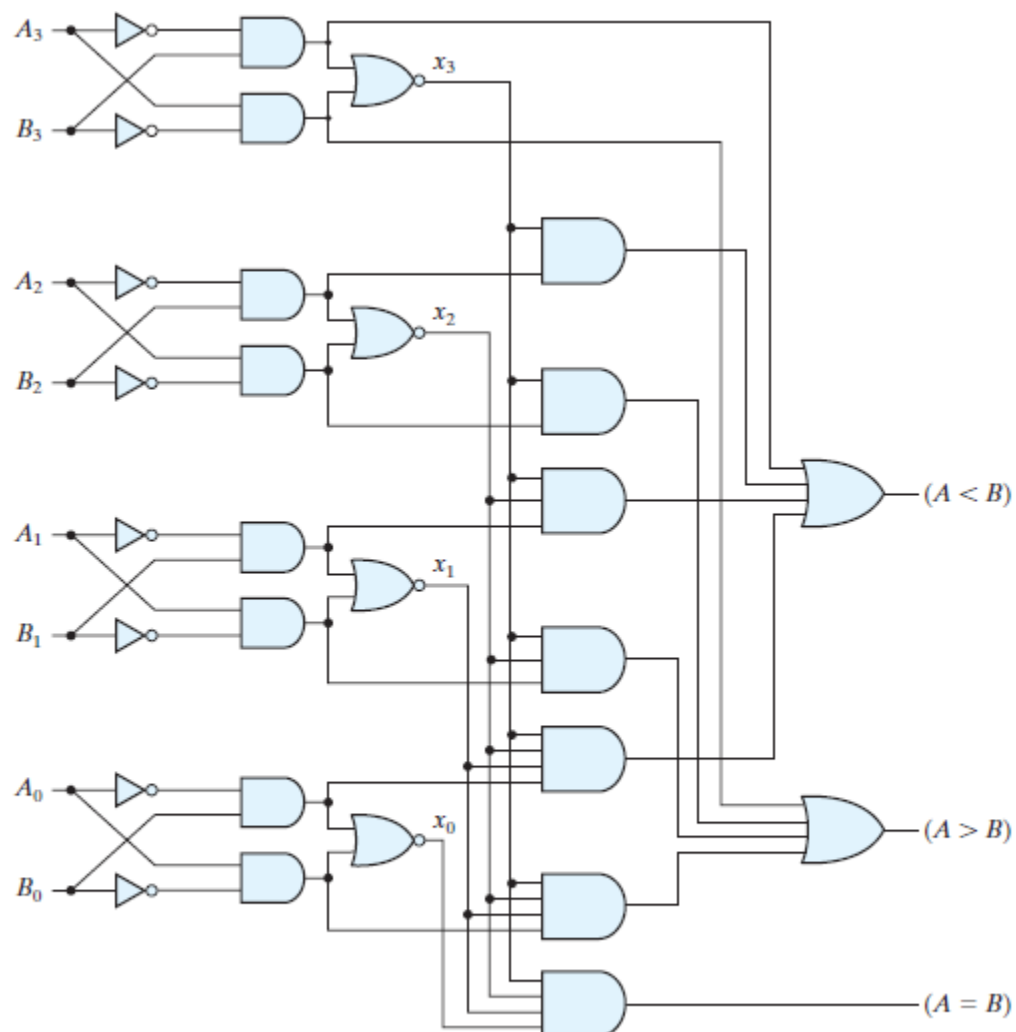
$$B = B_3 B_2 B_1 B_0$$

Function Equation

$$(A = B) = x_3 x_2 x_1 x_0$$

$$(A > B) = A_3 B'_3 + x_3 A_2 B'_2 + x_3 x_2 A_1 B'_1 + x_3 x_2 x_1 A_0 B'_0$$

$$(A < B) = A'_3 B_3 + x_3 A'_2 B_2 + x_3 x_2 A'_1 B_1 + x_3 x_2 x_1 A'_0 B_0$$



Four-bit magnitude comparator

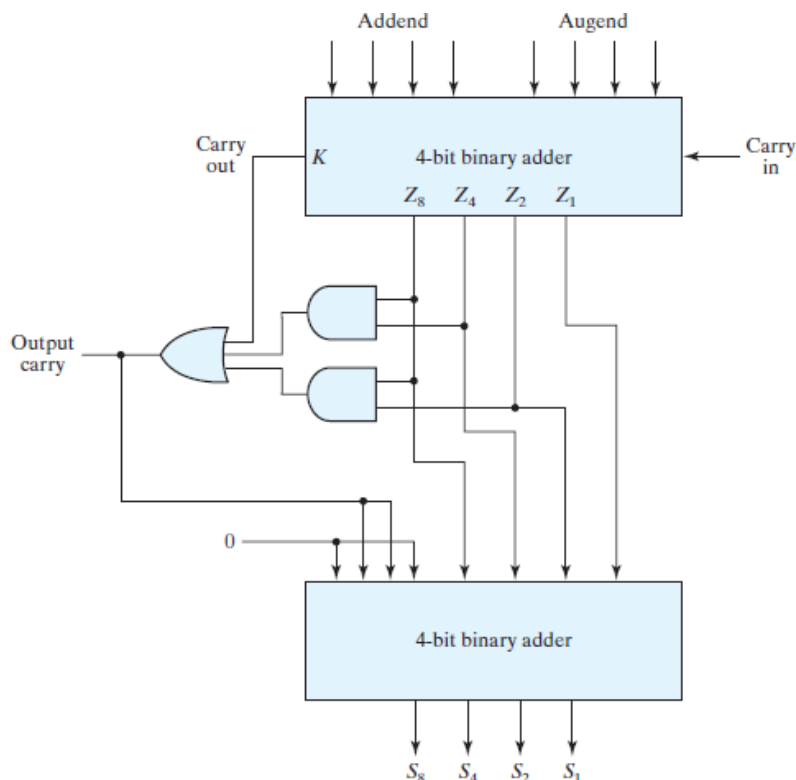
BCD Adder:

Design to perform BCD addition.(or) What is BCD adder? Design an adder to perform arithmetic addition of two decimal bits in BCD. (May -08)(Apr 2017,2018)[Nov – 2019]

- ❖ Consider the arithmetic addition of two decimal digits in BCD, together with an input carry from a previous stage. Since each input digit does not exceed 9, the output sum cannot be greater than $9 + 9 + 1$ = 19, the 1 in the sum being an input carry.
- ❖ Suppose we apply two BCD digits to a four-bit binary adder. The adder will form the sum in binary and produce a result that ranges from 0 through 19. These binary numbers are listed in Table and are labeled by symbols K, Z₈, Z₄, Z₂, and Z₁. K is the carry, and the subscripts under the letter Z represent the weights 8, 4, 2, and 1 that can be assigned to the four bits in the BCD code.

Derivation of BCD Adder

Binary Sum					BCD Sum					Decimal
K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

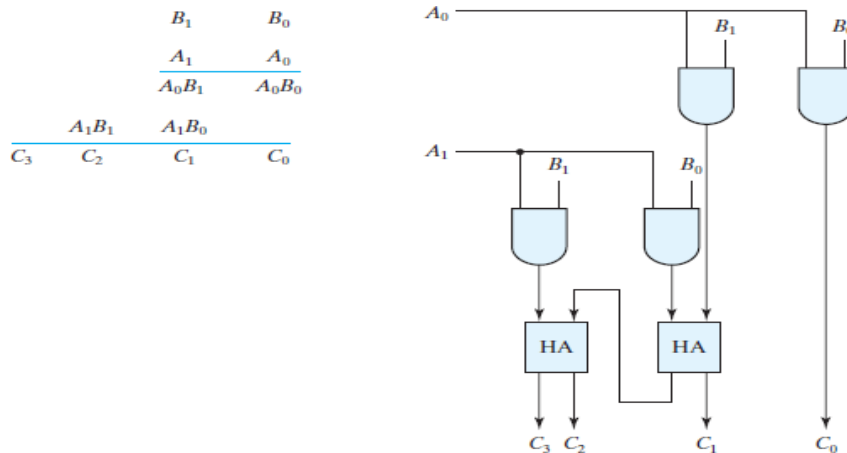


- ❖ A BCD adder that adds two BCD digits and produces a sum digit in BCD is shown in Fig. The two decimal digits, together with the input carry, are first added in the top four-bit adder to produce the binary sum.
 - ❖ When the output carry is equal to 0, nothing is added to the binary sum. When it is equal to 1, binary 0110 is added to the binary sum through the bottom four-bit adder.
 - ❖ The condition for a correction and an output carry can be expressed by the Boolean function
$$C = K + Z_8Z_4 + Z_8Z_2$$
 - ❖ The output carry generated from the bottom adder can be ignored, since it supplies information already available at the output carry terminal.
 - ❖ A decimal parallel adder that adds n decimal digits needs n BCD adder stages. The output carry from one stage must be connected to the input carry of the next higher order stage.
-

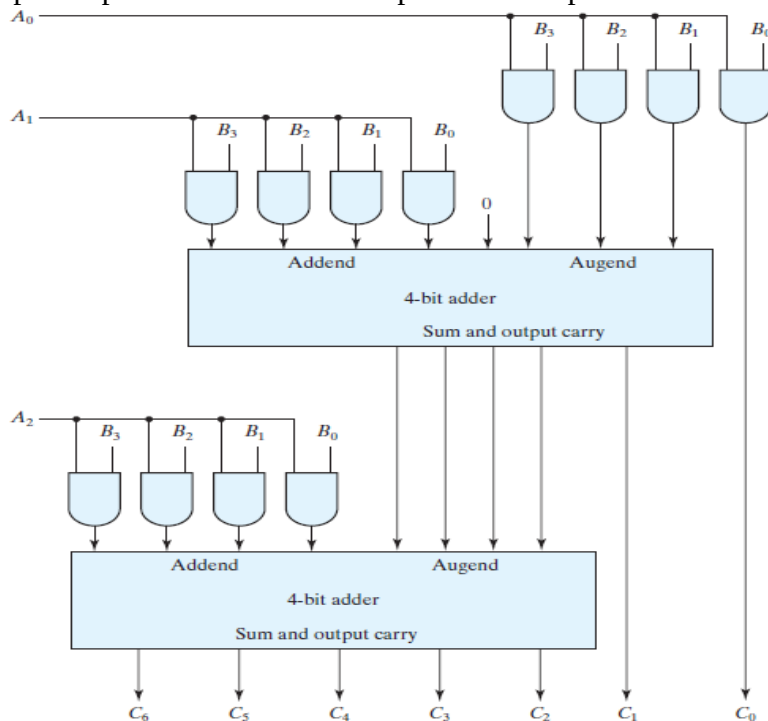
Binary Multiplier:

Explain about binary Multiplier.

- ❖ Multiplication of binary numbers is performed in the same way as multiplication of decimal numbers. The multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit. Each such multiplication forms a partial product.
- ❖ Successive partial products are shifted one position to the left. The final product is obtained from the sum of the partial products.



- ❖ A combinational circuit binary multiplier with more bits can be constructed in a similar fashion.
- ❖ A bit of the multiplier is ANDed with each bit of the multiplicand in as many levels as there are bits in the multiplier.
- ❖ The binary output in each level of AND gates is added with the partial product of the previous level to form a new partial product. The last level produces the product.



DECODERS AND ENCODERS

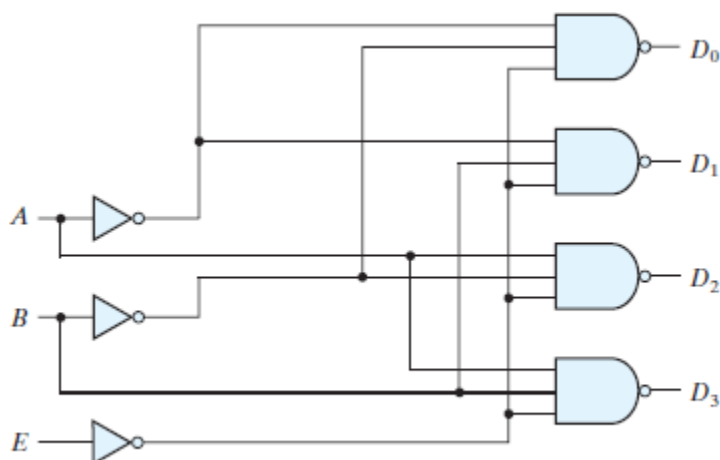
Decoder:

Explain about decoders with necessary diagrams.

(Apr 2018)(Nov 2018)

- ❖ A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines. If the n -bit coded information has unused combinations, the decoder may have fewer than 2^n outputs.
- ❖ The purpose of a decoder is to generate the 2^n (or fewer) minterms of n input variables, shown below for two input variables.

2 to 4 decoder:



(a) Logic diagram

E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	1

(b) Truth table

3 to 8 Decoder:

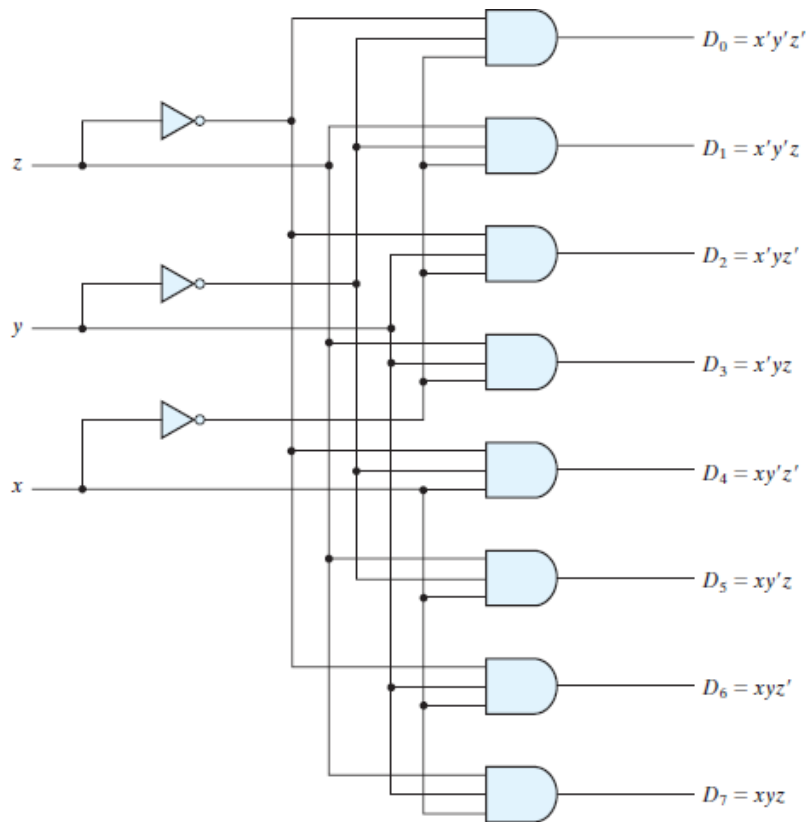
Design 3 to 8 line decoder with necessary diagram.

May -10)

Truth table:

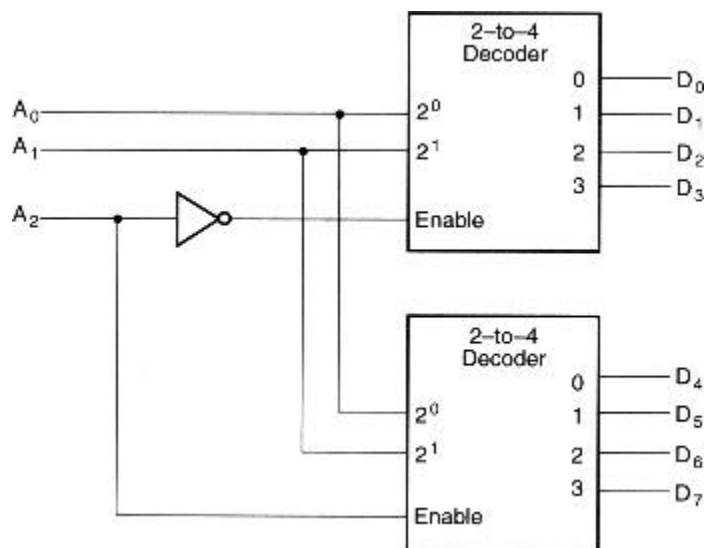
Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Logic diagram:



Design for 3 to 8 decoder with 2 to 4 decoder:

- ❖ Not that the two to four decoder design shown earlier, with its *enable* inputs can be used to build a three to eight decoder as follows.

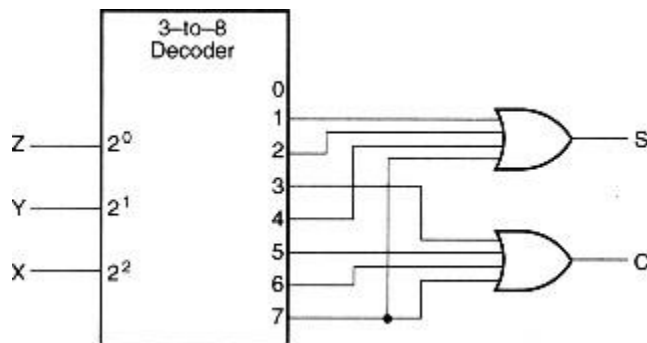


Implementation of Boolean function using decoder:

- ❖ Since the three to eight decoder provides all the minterms of three variables, the realisation of a function in terms of the sum of products can be achieved using a decoder and OR gates as follows.

Example: Implement full adder using decoder.

Sum is given by $\sum m(1, 2, 4, 7)$ while Carry is given by $\sum m(3, 5, 6, 7)$ as given by the minterms each of the OR gates are connected to.

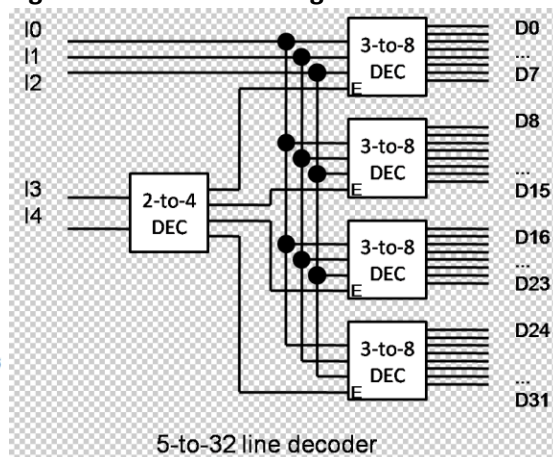
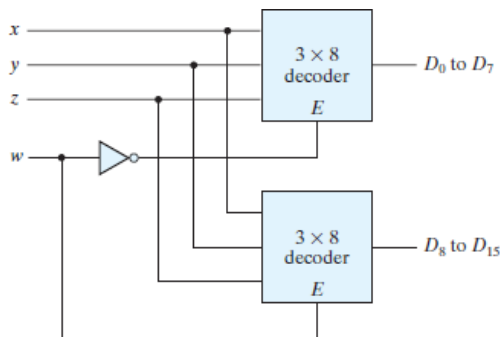


Solution :

Step 1 : Truth table

Inputs			Outputs	
A	B	C _{in}	Carry	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

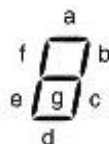
Design for 4 to 16 decoder using 3 to 8 decoder: **Design 5 to 32 decoder using 3 to 8 and 2 to 4 decoder:**



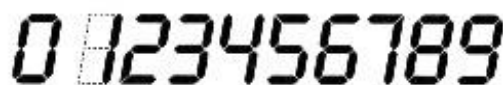
BCD to seven segment decoder

Design a BCD to seven segment code converter.

(May-06,10, Nov- 09)



(a) Segment designation



(b) Numeric designation for display

Truth table:

	BCD code				7-Segment code						
Digit	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

K-Map:

For (a)

	CD	00	01	11	10
AB	00	1	0	1	1
	01	0	1	1	1
	11	X	X	X	X
	10	1	1	X	X

$$a = A + C + BD + B'D'$$

For (b)

	CD	00	01	11	10
AB	00	1	1	1	1
	01	1	0	1	0
	11	X	X	X	X
	10	1	1	X	X

$$b = B' + C'D' + CD$$

For (c)

	CD	00	01	11	10
AB	00	1	1	1	0
	01	1	1	1	1
	11	X	X	X	X
	10	1	1	X	X

$$c = B + C' + D$$

For (d)

	CD	00	01	11	10
AB	00	1	0	1	1
	01	0	1	0	1
	11	X	X	X	X
	10	1	1	X	X

$$d = B'D' + CD' + BC'D + B'C + A$$

For (e)

	CD	00	01	11	10
AB	00	1	0	0	1
	01	0	0	0	1
	11	X	X	X	X
	10	1	0	X	X

$$e = B'D' + CD'$$

For (f)

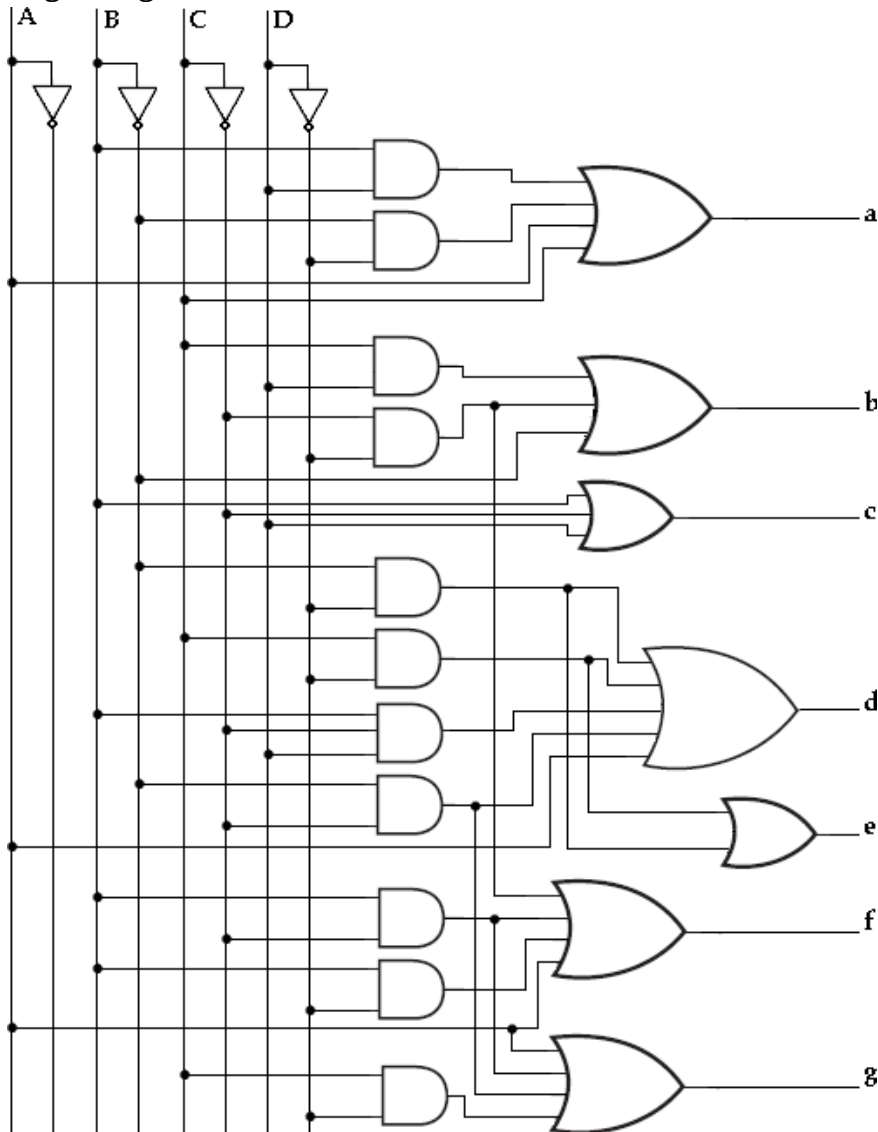
	CD	00	01	11	10
AB	00	1	0	0	0
	01	1	1	0	1
	11	X	X	X	X
	10	1	1	X	X

$$f = A + C'D' + BC' + BD'$$

AB \ CD		For (g)			
		00	01	11	10
00	0	0	1	1	
01	1	1	0	1	
11	X	X	X	X	
10	1	1	X	X	

$$g = A + BC' + B'C + CD'$$

Logic Diagram:



- ❖ The specification above requires that the output be zeroes (none of the segments are lighted up) when the input is not a BCD digit.
- ❖ In practical implementations, this may defer to allow representation of hexadecimal digits using these seven segments.

Encoder:

Explain about encoders. (Nov 2018)

- ❖ An encoder is a digital circuit that performs the inverse operation of a decoder. An encoder has 2^n (or fewer) input lines and n output lines. The output lines, as an aggregate, generate the binary code corresponding to the input value.

Octal to Binary Encoder:

- ❖ The encoder can be implemented with OR gates whose inputs are determined directly from the truth table. Output z is equal to 1 when the input octal digit is 1, 3, 5, or 7.
- ❖ Output y is 1 for octal digits 2, 3, 6, or 7, and output x is 1 for digits 4, 5, 6, or 7. These conditions can be expressed by the following Boolean output functions:

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

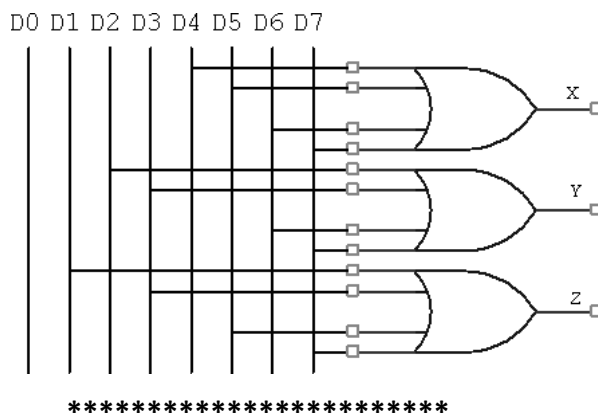
The encoder can be implemented with three OR gates.

Truth table:

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

- ❖ Another ambiguity in the octal-to-binary encoder is that an output with all 0's is generated when all the inputs are 0; but this output is the same as when D_0 is equal to 1. The discrepancy can be resolved by providing one more output to indicate whether at least one input is equal to 1.

Logic Diagram:



Priority Encoder:

Design a priority encoder with logic diagram.(or) Explain the logic diagram of a 4 – input priority encoder. (Apr – 2019)

A priority encoder is an encoder circuit that includes the priority function. The operation of the priority encoder is such that if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence.

Truth table:

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

Modified Truth table:

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	x	x	0
1	0	0	0	0	0	1
0	1	0	0	0	1	1
1	1	0	0			
0	0	1	0			
0	1	1	0	1	0	1
1	0	1	0			
1	1	1	0			
0	0	0	1			
0	0	1	1			
0	1	0	1			
0	1	1	1	1	1	1
1	0	0	1			
1	0	1	1			
1	1	0	1			
1	1	1	1			

K-Map:

$D_0D_1 \backslash D_2D_3$		For X			
		00	01	11	10
00	00	x	1	1	1
01	01	0	1	1	1
11	11	0	1	1	1
10	10	0	1	1	1

$$x = D_2 + D_3$$

$D_0D_1 \backslash D_2D_3$		For V			
		00	01	11	10
00	00	0	1	1	1
01	01	1	1	1	1
11	11	1	1	1	1
10	10	1	1	1	1

$$V = D_0 + D_1 + D_2 + D_3$$

$D_0D_1 \backslash D_2D_3$		For y			
		00	01	11	10
00	00	x	1	1	0
01	01	1	1	1	0
11	11	1	1	1	0
10	10	0	1	1	0

$$y = D_3 + D_1D_2$$

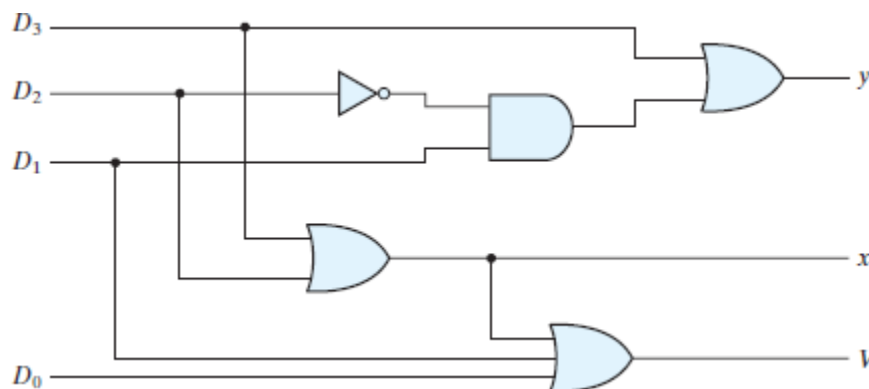
Logic Equations:

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

$$V = D_0 + D_1 + D_2 + D_3$$

Logic diagram:



MULTIPLEXERS AND DEMULTIPLEXERS

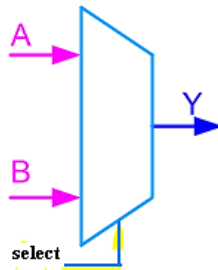
Multiplexer: (MUX)

Design a 2:1 and 4:1 multiplexer.

- ❖ A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines.
- ❖ Normally, there are 2^n input lines and n selection lines whose bit combinations determine which input is selected.

2 to 1 MUX:

A 2 to 1 line multiplexer is shown in figure below, each 2 input lines A to B is applied to one input of an AND gate. Selection lines S are decoded to select a particular AND gate. The truth table for the 2:1 mux is given in the table below.

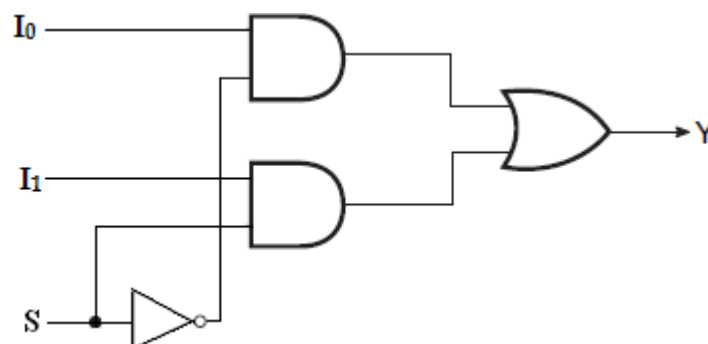


- ❖ To derive the gate level implementation of 2:1 mux we need to have truth table as shown in figure. And once we have the truth table, we can draw the K-map as shown in figure for all the cases when Y is equal to '1'.

Truth table:

S	Y
0	I_0
1	I_1

Logic Diagram:



4 to 1 MUX:

- ❖ A 4 to 1 line multiplexer is shown in figure below, each of 4 input lines I_0 to I_3 is applied to one input of an AND gate.

- ❖ Selection lines S_0 and S_1 are decoded to select one of the 4:1 mux is given in the t

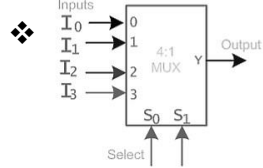
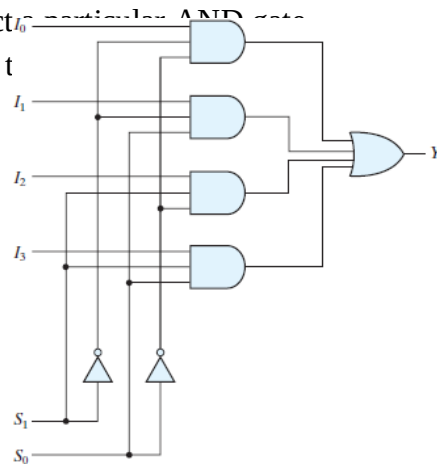


Diagram:



Truth Table:

SELECT INPUT		OUTPUT
S1	S0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Problems :

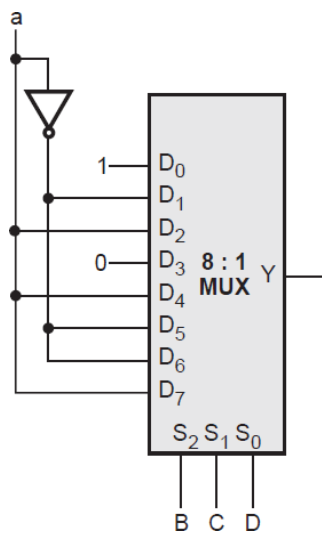
Example: Implement the Boolean expression using MUX

$$F(A,B,C,D) = \sum m(0,1,5,6,8,10,12,15)$$

(Apr 2017, Nov 2017)

Solution : Implementation table :

	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\bar{a}$	0	1	2	3	4	5	6	7
a	8	9	10	11	12	13	14	15
	1	$\bar{a}$	a	0	a	$\bar{a}$	$\bar{a}$	a



Example: Implement the boolean function using Multiplexer. [NOV – 2019]

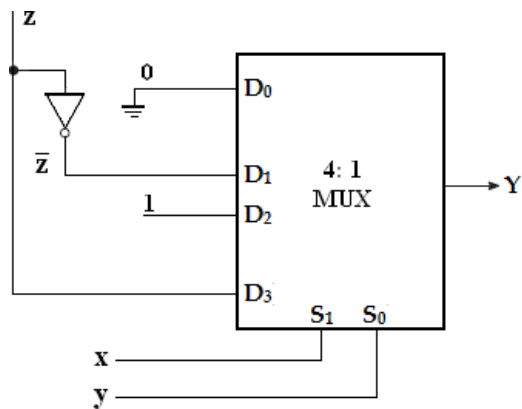
$$F(x, y, z) = \sum m(1, 2, 6, 7)$$

Solution:

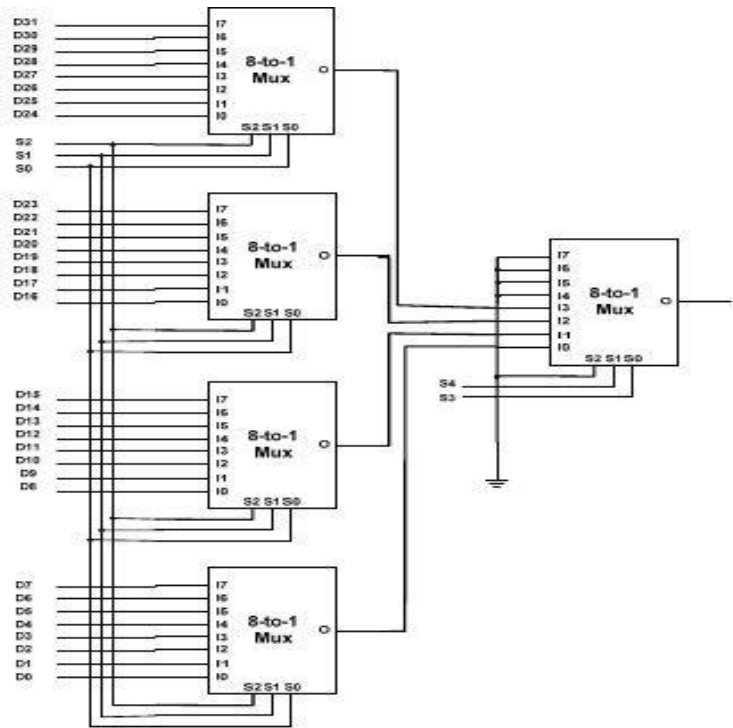
Implementation table:

	D ₀	D ₁	D ₂	D ₃
$\bar{z}$	0	1	2	3
z	4	5	6	7
	0	$\bar{z}$	1	z

Multiplexer Implementation:



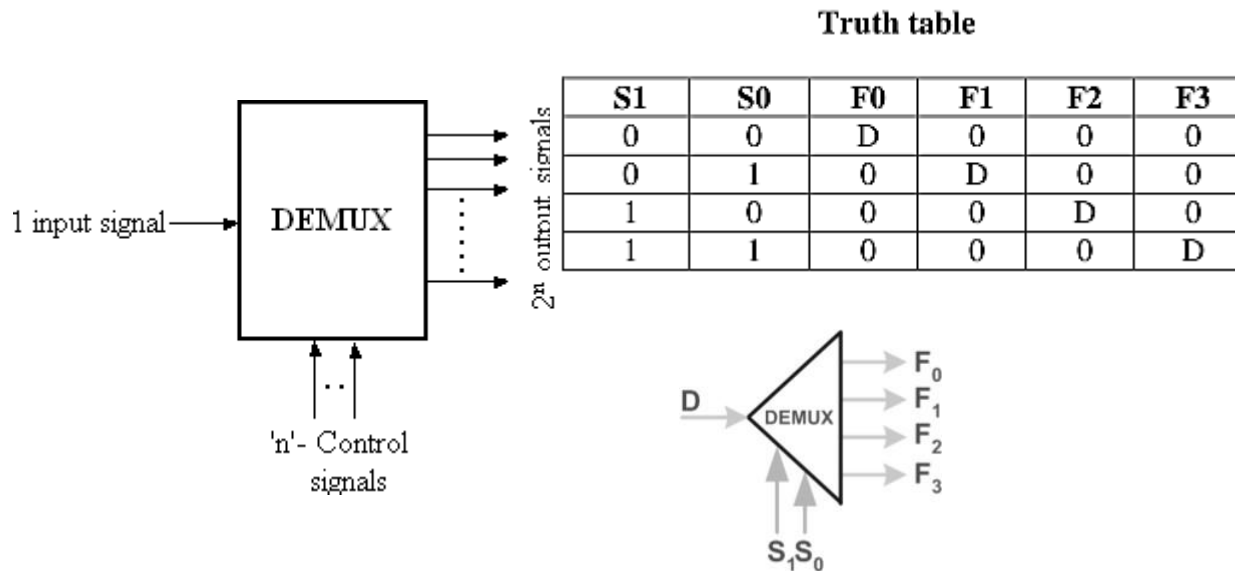
Example: 32:1 Multiplexer using 8:1 Mux (Nov 2018) (Apr – 2019)



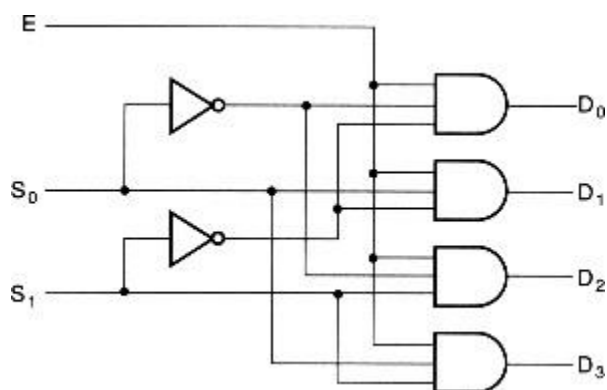
DEMULTIPLEXERS:

Explain about demultiplexers.

- ❖ The de- multiplexer performs the inverse function of a multiplexer, that is it receives information on oneline and transmits its onto one of 2^n possible output lines.
- ❖ The selection is by n input select lines. Example: 1-to-4 De- multiplexer



Logic Diagram:



Truth Table:

INPUT				OUTPUT			
E	D	S0	S1	Y0	Y1	Y2	Y3
1	1	0	0	1	0	0	0
1	1	0	1	0	1	0	0
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

Example:

1. Implement full adder using De-multiplexer.

Solution :

Step 1 : Truth table

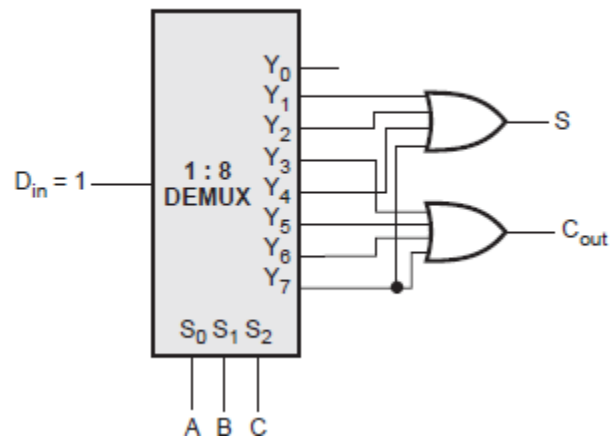
Inputs			Outputs	
A	B	C _{in}	Carry	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Step 2 : For full adder

$$\text{Carry} = C_{\text{out}} = \sum m(3, 5, 6, 7)$$

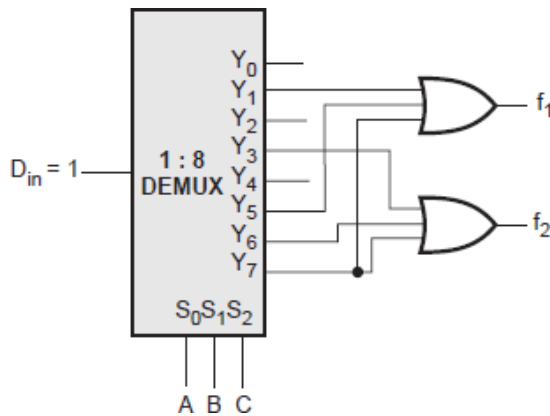
and $\text{Sum} = S = \sum m(1, 2, 4, 7)$

Step 3 : When $D_{\text{in}} = 1$, the demultiplexer gives minterms at the output.



2. Implement the following functions using de-multiplexer. $f_1(A,B,C) = \sum m(1,5,7)$, $f_2(A,B,C) = \sum m(3,6,7)$

Solution:



Parity Checker / Generator:

- ✓ A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even. The message, including the parity bit, is transmitted and then checked at the receiving end for errors. An error is detected if the checked parity does not correspond with the one transmitted.
- ✓ The circuit that generates the parity bit in the transmitter is called a *parity generator*. The circuit that checks the parity in the receiver is called a *parity checker*.
- ✓ In even parity system, the parity bit is '0' if there are even number of 1s in the data and the parity bit is '1' if there are odd number of 1s in the data.
- ✓ In odd parity system, the parity bit is '1' if there are even number of 1s in the data and the parity bit is '0' if there are odd number of 1s in the data.

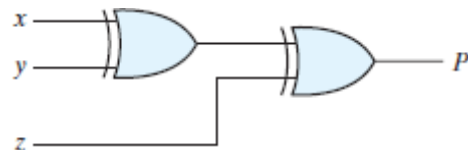
3-bit

Four Bits Received				Parity Error Check
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>	<i>C</i>
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Even
Parity
generato
r: Truth
Table:

$$P = x \oplus y \oplus z$$

Logic Diagram:



4-bit Even parity checker:
Truth Table:

$$C = x \oplus y \oplus z \oplus P$$

Logic Diagram:

